

Person 3 — UI, GraphRAG Docs & Demo Lead

Experience Lead: analyst dashboard, policy document prep, QA of answer files, demo video, blog + social post

The Mission

Build an agentic fraud investigation system on TigerGraph that takes an uncertain fraud signal (risk score, customer report, or analyst request), investigates it against the IEEE-CIS transaction graph, decides whether it has enough evidence to act, and — if not — takes controlled steps to get more. It must create/progress a case, recommend or take next-best actions inside policy limits, explain its reasoning, and remember past cases to improve future ones. Deliverable: agent + repo + 20 benchmark case answers + demo video + blog post + social post, by **Sept 24, 2026, 11:59 PM IST**.

Why This Solution Should Stand Out

- **Confidence is visible, not a black box.** Every case shows a running confidence score built from named, weighted components (bank risk score, typology match strength, similarity to past fraud, evidence coverage) — judges can see exactly why the agent is or isn't sure, which speaks directly to the 25%-weighted "Next best action" criterion.
- **Memory lives inside the graph, not bolted on.** Past cases are vertices with a SIMILAR_TO edge computed from embeddings, so "case memory" is literally a graph traversal — a much stronger TigerGraph story than a side Pinecone/Chroma DB most teams will reach for.
- **Every recommendation cites its evidence and its policy clause.** No generic "this looks suspicious" — output reads like an auditor wrote it, with evidence IDs and policy clause IDs inline. This is what "Case summary & explainability" is graded on.
- **A real approval workflow, not a toggle.** High-impact actions (block, freeze, file SAR) sit in a pending queue until a human clicks approve in the UI. This is the most literal, demoable way to satisfy "operate within predefined policies and permissions."
- **The whole thing is free and runs on a laptop.** No paid API, no paid hosting, nothing that can rate-limit or lock the team out mid-demo.

Tech Stack (100% Free)

Layer	Tool	Why it's free / notes
Graph database	TigerGraph Savanna (cloud) — enable auto-stop/auto-start. Fallback: TigerGraph Community Edition via Docker if Savanna signup is slow.	Both free for the hackathon (per challenge brief).
Graph algorithms	Built-in TigerGraph GDS library (Weakly Connected Components, Jaccard similarity) + custom GSQL	Included with TigerGraph, no extra cost.
Graph ↔ agent bridge	tigergraph-mcp (open source)	github.com/tigergraph/tigergraph-mcp , free.
LLM — primary	Groq API (Llama 3.3 70B / 3.1 8B)	Free tier, no card, fast — good for iterating live during judging.
LLM — backup	Google Gemini 2.0/2.5 Flash (AI Studio)	Generous free daily quota, switch to this if Groq throttles.
Embeddings	sentence-transformers (all-MiniLM-L6-v2), run locally	Free, offline, zero rate limit — safest choice for a live demo.

Layer	Tool	Why it's free / notes
Vector search	TigerGraph native vector index (preferred)	Fallback: FAISS/Chroma locally if native vector search is slow to set up.
Agent framework	LangGraph	Open source; models the 8-step flow as an explicit graph — good for the "agentic design" score.
Mock action APIs	Small FastAPI/Python stub service + JSON or SQLite audit ledger	Free; simulates block/freeze/refund/CRM/SAR per brief's "may be simulated" allowance.
UI	Streamlit	Free, fast to build, runs locally.
Graph visualization	streamlit-agraph or pyvis	Free.
Version control	GitHub	Free (public or private repo).
Demo hosting (optional stretch)	Streamlit Community Cloud	Free, only if time allows a shareable link.

Graph Data Model (draft — confirm against dataset README)

Vertices: Customer, Account, Card, Device, Address, EmailDomain, Transaction, ProductCategory (merchant proxy), FraudCase, Evidence, PolicyClause, FraudTypology, Action, ApprovalDecision.

Edges: Customer■OWNS→Account, Account■HOLDS→Card, Account■MADE→Transaction, Transaction■USES→Card/Device/Address/EmailDomain, Transaction■TO→ProductCategory, Account■SHARES_DEVICE/SHARES_EMAIL/SHARES_ADDRESS→Account (derived, self-join — this is the fraud-ring backbone), FraudCase■INVOLVES→Transaction/Account/Customer, FraudCase■HAS_EVIDENCE→Evidence, FraudCase■CITES→PolicyClause, FraudCase■MATCHES→FraudTypology, FraudCase■RECOMMENDS→Action, FraudCase■SIMILAR_TO→FraudCase (embedding similarity — the case-memory edge).

This is a starting draft from the known IEEE-CIS column structure. Confirm exact fields against the dataset README in the first hour — do not build against assumptions past that point.

Investigation Flow → Agent Nodes

- **1. trigger_intake** — normalize a risk score / customer report / analyst request into a common event.
- **2. open_or_load_case** — check the graph for an existing open case on this entity; else create one.
- **3. gather_graph_evidence** — parallel GSQL/MCP calls: transaction history, shared device/IP/email links, velocity checks.
- **4. gather_graphrag_context** — retrieve relevant policy clauses, fraud typologies, and similar prior cases (structured, not raw dump).
- **5. assess_patterns_and_risk** — match against known typologies, compute the weighted confidence score.
- **6. uncertainty_gate** — confidence $\geq$ threshold? If not, branch to requesting more evidence (step-up auth, account-owner validation, analyst input) and loop back to step 3.
- **7. decide_actions** — pick next-best action(s), constrained by the policy table.
- **8. approval_gate** — hold for human approval if the action requires it; otherwise proceed.
- **9. execute_or_record** — call the mock action API and log to the audit ledger.
- **10. explain** — generate the narrative, citing evidence IDs and policy clause IDs.
- **11. update_case_memory** — write the case back to the graph, including new SIMILAR_TO edges.

Two-Day Timeline

Hours are **elapsed hours from kickoff**, not clock times — map them onto whatever the actual venue schedule is (including sleep, if any). The shape matters more than the exact numbers: contracts first, parallel build against mocks, integrate early, freeze features with a third of the time left for polish, video, and writeups.

When	Milestone	Owner(s)
H0–H1	Kickoff together: read the dataset README, finalize JSON contracts (case schema, MCP tool I/O), create shared GitHub repo, get API keys (TigerGraph Savanna, Groq, Gemini).	Everyone
H1–H2	P1 publishes MCP tool I/O shapes. P2 publishes the case-record JSON schema. Both go in the repo /contracts folder.	P1 + P2
H1–H6	Core build sprint 1 — everyone works against the published contracts / mocks in parallel (see your task list).	Everyone
H6–H7	Sync #1 — demo current state to each other, unblock issues.	Everyone
H7–H10	Core build sprint 2 + first integration: P2 swaps mock tools for P1's live MCP tools.	Everyone
H10–H12	End-to-end test on 2–3 sample cases; log gaps to fix on Day 2.	Everyone
H12–H15	Fix integration gaps. P1 tunes graph algorithms on real gaps found. P2 hardens confidence/approval logic. P3 wires UI to the live agent.	Everyone
H15–H18	Run all 20 benchmark cases; generate + spot-check answer files; verify case-memory reuse across cases.	Everyone
H18–H20	Polish: UI states, explanation quality, edge cases (no fraud found, insufficient evidence, SAR-required).	Everyone
H20–H21	Record the 3–5 min demo video together (storyboard first).	Everyone
H21–H22	Write blog post + social post + finalize README.	P3
H22–H23	Buffer: bug bash, re-check API rate limits, re-run any failed cases.	Everyone
H23	Submit — well ahead of the deadline.	Everyone

Rows highlighted in your color are the ones you personally own or co-own.

Judging Criteria Alignment

Criterion	Weight	Mainly driven by
Investigation accuracy	25%	P1 (graph pattern quality) + P2 (pattern-matching / risk logic)
Next best action	25%	P2 (confidence engine, policy engine, evidence-driven updates)
Case summary & explainability	10%	P2 (explanation generation) + P3 (how it's presented)
Agentic design & engineering	15%	P2 (orchestration, tools, memory) + P1 (tool/graph design)
Innovation	15%	All three — graph-native memory, transparent confidence, approval workflow
Demo quality & completeness	10%	P3

Your Role in Detail

- Own the Streamlit analyst dashboard: case queue, case detail, evidence timeline, confidence view, graph visualization, approval workflow.
- Own cleaning and chunking the fraud policy / typology / regulatory reference documents so P1 can load them as PolicyClause / FraudTypology vertices.
- Own final QA of the 20 answer files against the dataset README's exact required format.
- Own the demo video, technical blog post, and social post.

Your Day 1 Task List

- Join the H0–H1 kickoff and help finalize the case-record JSON schema — this is exactly what the UI will render, so your input on what's easy/hard to display matters here.
- Chunk and clean the fraud policy / typology / regulatory reference documents into clear sections with stable IDs (e.g. POL-001, TYP-003) — this is the raw material for PolicyClause and FraudTypology vertices and for GraphRAG citation. Deliver this to P1 by roughly H6.
- Scaffold the Streamlit app against MOCK case JSON (matching P2's published schema) — do not wait for the live agent: case queue list view, case detail view with an evidence timeline and a confidence gauge / traffic-light indicator, and a recommended-actions panel.
- Add the graph visualization component (streamlit-agraph or pyvis) that renders the relevant subgraph (accounts, devices, transactions) for a selected case.

Your Day 2 Task List

- Wire the UI to P2's live agent output, replacing the mock JSON.
- Build the human-in-the-loop approval UI: a pending-actions queue with approve/reject buttons that calls P2's API and updates the audit ledger.
- If time allows, add a conversational panel where an analyst can ask the agent follow-up questions about a case — treat this as the first thing to cut if the team is behind schedule.
- QA pass: verify all 20 generated answer files match the dataset README's required format field-by-field; flag any gaps to P2 immediately rather than at the end.
- Storyboard and record the 3–5 minute demo video with the team: trigger → evidence gathering → uncertainty → requesting more evidence → action → explanation → a second, related case showing memory reuse. Assign who drives the UI, who narrates, who operates the recording.
- Write the technical blog post (what you built, architecture, how TigerGraph is used, agentic capabilities implemented, what you learned, what you'd improve) — start drafting in the afternoon so it isn't rushed; pull screenshots from the working UI.
- Publish the social post on X or LinkedIn tagging @TigerGraphDB, linking the blog post/demo.

Definition of Done

- Streamlit app renders live case data: evidence timeline, confidence/uncertainty visualization, graph view, recommended next-best actions, and a working approval flow for gated actions.
- All 20 answer files verified against the dataset README's exact required format, with any gaps reported back and fixed.
- 3–5 minute demo video recorded and ready to submit.
- Blog post drafted covering all six required sections.
- Social post published, tagging @TigerGraphDB, with a link to the blog/demo.

Core (must-ship) vs. Stretch (cut first if behind)

Core — must ship	Stretch — nice to have
• Case queue + case detail view (evidence, confidence, recommended actions). • Graph visualization of the relevant subgraph per case. • Approval workflow UI. • QA of all 20 answer files against the README format. • Demo video, blog post, social post.	• Conversational chat panel for analyst follow-up questions. • Deployed public Streamlit Community Cloud link. • Custom theming/branding polish.

Dependencies

You depend on: P2's case-record JSON schema (published at H1) so you can build against realistic mock data immediately, and P2's live agent output (target ~H15) to fully wire the dashboard. Your own policy-document cleanup is on P1's critical path — aim to hand it off by H6 so it doesn't stall their PolicyClause/FraudTypology loading.

Others depend on you for: P1 is waiting on your cleaned/chunked policy and typology documents (target handoff ~H6) to finish loading PolicyClause and FraudTypology vertices. Nobody is blocked on your UI work itself, which gives you the most schedule slack of the three roles — use it to make explainability and the demo genuinely polished, since those map directly to two graded criteria.

Free Resources & Links for Your Role

- Streamlit docs: <https://docs.streamlit.io>
- streamlit-agraph: <https://github.com/ChrisDelClea/streamlit-agraph>
- pyvis: <https://pyvis.readthedocs.io>
- Streamlit Community Cloud (free deploy, optional stretch): <https://streamlit.io/cloud>

Risks & Contingencies (whole team should know these)

- **Free-tier LLM rate limits.** Don't hammer the API while iterating on prompts — use fixture responses during early development, switch to Gemini if Groq throttles, and test on 2–3 cases before running all 20.
- **Dataset README not read carefully enough.** The exact answer-file format, column meanings, and case format all come from it — the whole team reads it together in the first hour before writing any schema.
- **Savanna signup friction.** If TigerGraph Savanna account creation is slow, fall back to Community Edition in Docker immediately rather than losing build hours waiting.
- **Integration left too late.** The single biggest risk in a 3-person split is each person's piece working alone and failing to connect. The published contracts (H1) and the two scheduled integration checkpoints (H7, H15) exist specifically to prevent this — do not skip them even if your own piece feels unfinished.

Full Submission Checklist

- Working agent (code in the repo)
- GitHub repository, public or shared with judges
- One answer file per benchmark case (20 total): case record, evidence, findings, decisions, actions taken
- Each case also written into the graph
- Suspicious Activity Report generated when policy requires it
- Next-best action + required approval route recorded both before and after any additional evidence is gathered
- 3–5 minute demo video showing the agent end to end
- Technical blog post: what you built, architecture, how TigerGraph is used, agentic capabilities, what you learned, what you'd improve
- Social post on X or LinkedIn tagging @TigerGraphDB, linking the blog/demo

Submit at: <https://forms.gle/yxXzqSULGgZ9VUF56> — one submission per team, by the team lead, by Sept 24, 2026, 11:59 PM IST. No resubmissions. Support: TigerGraph Discord <https://discord.gg/7JmKCAy9D3>.